

MP2: Frame Manager

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

Main: Completed.

System Design

Overview

The project implements a **contiguous frame memory manager** designed to efficiently allocate and release memory frames within a 32MB physical memory space. Memory is divided into distinct regions to separate kernel operations from user processes, ensuring safety and isolation. Each memory frame has a fixed size of 4KB.

Memory Layout

The memory is organized as follows:

- **Reserved Memory:** The first 2MB is reserved and unavailable for allocation.
- **Kernel Frame Pool:** The next 2MB (2MB–4MB) is dedicated to the kernel, allowing it to manage its own memory.
- **Process Frame Pool:** The remaining 28MB (4MB–32MB) is used by user processes and managed separately.

Allocation requests are handled within the corresponding pool, providing **contiguous frames** for efficient memory usage. Released frames are returned to their respective pools, maintaining separation between kernel and user space.

Frame Tracking Using Bitmap

A bitmap-based approach is employed to track the status of each frame:

- **2-bit Representation:** Each frame uses two bits to indicate whether it is **free**, **allocated**, or the **head** of a contiguous allocation.
- **In-pool Storage:** The bitmap is stored within the pool to minimize memory overhead.
- **Inaccessible Frames:** Certain frames, such as memory holes or reserved regions, are marked inaccessible and never allocated.

ContFramePool Class

The `cont_frame_pool` class manages all operations related to a memory pool. Its main responsibilities include:

- `get_frames(n)`: Allocates `n` contiguous frames.
- `release_frames(frame)`: Frees previously allocated frames.
- `mark_inaccessible(start, size)`: Marks frames that must remain unavailable.
- `needed_info_frames(total_frames)`: Computes the number of metadata frames required to manage the pool efficiently.

Allocation Algorithm and Testing

The memory manager ensures correctness and efficiency through the following mechanisms:

1. Allocation searches for contiguous free frames, skipping inaccessible regions.
2. Released frames are returned to the pool and marked free in the bitmap.
3. Recursive memory tests write unique values to allocated frames and verify them before releasing, ensuring integrity.

Key Features

- Prevents allocation of reserved or inaccessible memory regions.
- Handles special cases like zero-frame or maximum-frame requests gracefully.
- Stores metadata efficiently within the memory pool to minimize overhead.

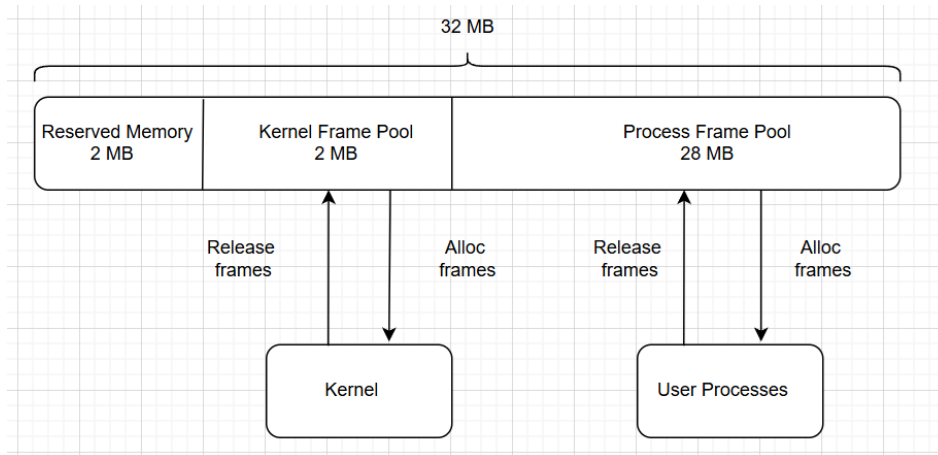


Figure 1: Memory Frame Manager Structure

Code Description

For this machine problem, we modified `cont_frame_pool.H` and `cont_frame_pool.C` to implement a frame manager responsible for efficiently allocating and releasing contiguous memory frames. The system uses a bitmap to monitor the state of each frame, ensuring proper allocation, deallocation, and marking of frames as inaccessible when required.

FrameState Enumeration

The `FrameState` enumeration is used to track the state of each frame and defines the possible conditions of memory allocation:

- **Free:** The frame is available for allocation.
- **Allocated:** The frame is currently in use.
- **HeadOfSequence (HoS):** Marks the first frame in a contiguous block of allocated frames.

This categorization allows efficient memory management and ensures that contiguous allocations are properly tracked.

Compilation and Execution

To compile the modified code, we use the existing Makefile:

```
make clean && make
```

To execute the kernel in QEMU:

```
qemu-system-x86_64 -kernel kernel.bin
```

`cont_frame_pool.C`: Constructor

The constructor initializes the frame pool and sets up the bitmap for memory allocation:

- Stores a reference to the current instance in the pool pointer.
- Initializes key variables such as `baseFrameNo`, `nFrames`, `infoFrameNo`, and `nFreeFrames`.
- Determines the bitmap storage location: if `infoFrameNo` is 0, the bitmap is placed in the first frame of the pool; otherwise, it is stored at the specified `infoFrameNo`.
- Initializes all frames as **Free** and marks the metadata frame as **HeadOfSequence** (HoS), decrementing the free frame count accordingly.

```

/*-----*/
/* CONSTRUCTOR */
/*-----*/
ContFramePool::ContFramePool(unsigned long _base_frame_no,
                             unsigned long _n_frames,
                             unsigned long _info_frame_no)
{
    base_frame_no = _base_frame_no;
    nframes = _n_frames;
    info_frame_no = _info_frame_no;

    // Calculate how many frames needed for bitmap
    unsigned long needed = ContFramePool::needed_info_frames(nframes);

    if (info_frame_no == 0) {
        info_frame_no = base_frame_no;
    }

    assert((info_frame_no >= base_frame_no) &&
           ((info_frame_no + needed) <= (base_frame_no + nframes)));

    // Bitmap location (physical-to-pointer assumption as in starter code)
    bitmap = (unsigned char*)(info_frame_no * FRAME_SIZE);

    // Initialize all frames as Free
    for (unsigned long i = 0; i < nframes; i++) {
        set_state(i, FrameState::Free);
    }

    // Reserve all info frames (mark as Used so they won't be allocated)
    unsigned long rel = info_frame_no - base_frame_no;
    for (unsigned long i = 0; i < needed; i++) {
        set_state(rel + i, FrameState::Used); // <-- use 'Used' per header
    }

    // Update free count
    nFreeFrames = (nframes >= needed) ? (nframes - needed) : 0;

    // Link into global pool list
    this->next_pool = pool_list_head;
    pool_list_head = this;

    Console::puts("ContFramePool initialized.\n");
}

```

Figure 2: ContFramePool() Constructor Implementation

cont_frame_pool.C: get_frames()

This method searches for a contiguous block of *n* free frames. If found:

- Marks the first frame as HoS.
- Marks the remaining frames as Allocated.
- Returns the starting frame number of the allocated block.

```

/*-----*/
/* ALLOCATE CONTIGUOUS FRAMES */
/*-----*/
unsigned long ContFramePool::get_frames(unsigned int _n_frames) {
    if (_n_frames == 0) return 0;
    if ((unsigned long)_n_frames > nFreeFrames) return 0;

    if ((unsigned long)_n_frames > nframes) return 0;
    unsigned long limit = nframes - _n_frames + 1;

    for (unsigned long i = 0; i < limit; i++) {
        if (get_state(i) != FrameState::Free) continue;

        bool ok = true;
        for (unsigned long j = 0; j < _n_frames; j++) {
            if (get_state(i + j) != FrameState::Free) {
                ok = false;
                break;
            }
        }
        if (!ok) continue;

        // Mark first frame as HoS (Head of Sequence)
        set_state(i, FrameState::HoS); // <-- use 'HoS' per header
        // Mark remaining frames as Used
        for (unsigned long j = 1; j < _n_frames; j++) {
            set_state(i + j, FrameState::Used);
        }

        nFreeFrames -= _n_frames;
        return base_frame_no + i;
    }

    return 0; // not found
}

```

Figure 3: get_frames() Implementation

cont_frame_pool.C: release_frames()

This method releases previously allocated frames:

- Identifies the correct memory pool and validates that the first frame is marked as HoS.
- Iterates through the contiguous block, marking each frame as **Free**.

```

/*-----*/
/* RELEASE FRAMES (STATIC, search pool) */
/*-----*/
void ContFramePool::release_frames(unsigned long _first_frame_no) {
    ContFramePool* p = pool_list_head;
    while (p != nullptr) {
        if ((_first_frame_no >= p->base_frame_no) &&
            (_first_frame_no < (p->base_frame_no + p->nframes))) {

            unsigned long idx = _first_frame_no - p->base_frame_no;
            // Ensure it's a head of sequence
            assert(p->get_state(idx) == FrameState::HoS);

            // Free the head
            p->set_state(idx, FrameState::Free);
            p->nFreeFrames++;

            // Free following Used frames until Free or HoS
            for (unsigned long i = idx + 1; i < p->nframes; i++) {
                FrameState s = p->get_state(i);
                if (s == FrameState::Free || s == FrameState::HoS) break;
                p->set_state(i, FrameState::Free);
                p->nFreeFrames++;
            }
            return;
        }
        p = p->next_pool;
    }

    Console::puts("ContFramePool::release_frames: invalid frame\n");
    assert(false);
}

```

Figure 4: release_frames() Implementation

cont_frame_pool.C: mark_inaccessible()

This method prevents memory allocation in specific regions:

- Marks a range of frames as inaccessible in the bitmap to ensure they are never allocated.

```

/*-----*/
/* MARK FRAMES INACCESSIBLE */
/*-----*/
void ContFramePool::mark_inaccessible(unsigned long _base_frame_no,
    unsigned long _n_frames) {
    unsigned long start = _base_frame_no - base_frame_no;
    assert(start < nframes);
    assert((start + _n_frames) <= nframes);

    // Mark first as HoS, rest as Used
    set_state(start, FrameState::HoS);
    for (unsigned long i = 1; i < _n_frames; i++) {
        set_state(start + i, FrameState::Used);
    }

    if (_n_frames <= nFreeFrames) nFreeFrames -= _n_frames;
    else nFreeFrames = 0;
}

```

Figure 5: mark_inaccessible() Implementation

cont_frame_pool.C: needed_info_frames()

This function calculates the number of frames required to store the management bitmap:

- Determines the space required based on the number of frames.
- Ensures that the allocation fits within the pool.

```
/*-----*/
/* CALCULATE INFO FRAMES NEEDED */
/*-----*/
unsigned long ContFramePool::needed_info_frames(unsigned long _n_frames) {
    if (_n_frames == 0) return 0;

    unsigned long total_bits = _n_frames * 2UL;
    unsigned long total_bytes = total_bits / 8UL;
    if (total_bits % 8UL) total_bytes++;

    unsigned long frames = total_bytes / FRAME_SIZE;
    if (total_bytes % FRAME_SIZE) frames++;

    return frames;
}
```

Figure 6: needed_info_frames() Implementation

Helper Functions: get_state() and set_state()

The helper functions manage frame states:

- get_state() reads the state of a frame from the bitmap.
- set_state() updates the bitmap to reflect a new state.

```
/*-----*/
/* BITMAP HELPERS (2 bits per frame) */
/*-----*/
ContFramePool::FrameState ContFramePool::get_state(unsigned long _frame_no) {
    unsigned long bit_index = _frame_no * 2;
    unsigned long byte_index = bit_index / 8;
    unsigned long offset = bit_index % 8;

    unsigned char state = (bitmap[byte_index] >> offset) & 0b11u;
    return static_cast<FrameState>(state);
}

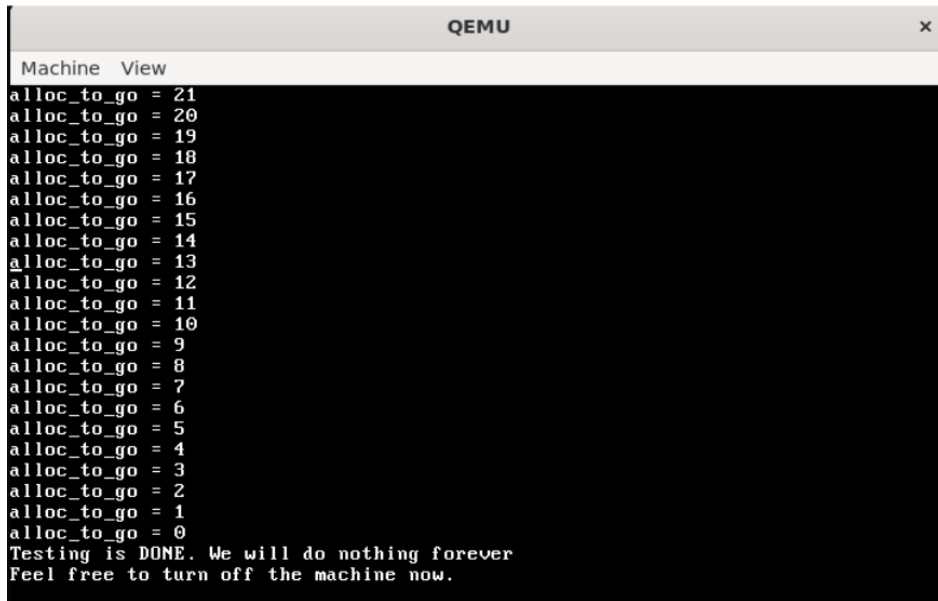
void ContFramePool::set_state(unsigned long _frame_no, FrameState _state) {
    unsigned long bit_index = _frame_no * 2;
    unsigned long byte_index = bit_index / 8;
    unsigned long offset = bit_index % 8;

    // clear 2 bits at position
    bitmap[byte_index] &= static_cast<unsigned char>(~(0b11u << offset));
    // set new state
    bitmap[byte_index] |= static_cast<unsigned char>((static_cast<unsigned char>(_state) & 0b11u) << offset);
}
```

Figure 7: get_state() & set_state() Implementation

Output

The system successfully allocates and releases frames as expected. An example of frame allocation and deallocation output can be observed in QEMU.



```
QEMU
Machine View
alloc_to_go = 21
alloc_to_go = 20
alloc_to_go = 19
alloc_to_go = 18
alloc_to_go = 17
alloc_to_go = 16
alloc_to_go = 15
alloc_to_go = 14
alloc_to_go = 13
alloc_to_go = 12
alloc_to_go = 11
alloc_to_go = 10
alloc_to_go = 9
alloc_to_go = 8
alloc_to_go = 7
alloc_to_go = 6
alloc_to_go = 5
alloc_to_go = 4
alloc_to_go = 3
alloc_to_go = 2
alloc_to_go = 1
alloc_to_go = 0
Testing is DONE. We will do nothing forever
Feel free to turn off the machine now.
```

Figure 8: Output

Testing

To validate the functionality and reliability of the `cont_frame_pool`, we performed comprehensive testing. In addition to the built-in test memory functions, we implemented custom unit tests in `kernel.C` to examine the behavior of each method individually.

The key test cases include:

- **Frame Allocation and Release:** Checks that frames can be allocated and freed correctly, updating the bitmap and free frame count as expected.
- **Marking Inaccessible Frames:** Ensures that frames marked as inaccessible cannot be allocated, protecting reserved memory regions.
- **Calculation of Info Frames:** Verifies that the number of frames required to store the management bitmap is correctly computed.
- **Edge Case Handling:** Tests scenarios such as allocating zero frames or requesting more frames than available, ensuring graceful failure and correct system behavior.

Compilation and Execution: The tests were built and run using the following commands:

```
make clean && make
make run
```

All test cases passed successfully, confirming that: - Allocated frames were correctly tracked, - Inaccessible frames remained protected, and - Edge cases were handled properly without causing errors.

After confirming the correctness of the implementation, the additional test cases were commented out to maintain a clean final submission.

```
phaniyoth11@PhaniGowthar: ~/mp2/MP2_Sources/MP2_Sources$ make run
Linux
qemu-system-x86_64 -kernel kernel.bin -serial stdio
ContFramePool initialized.
Hello World!
alloc_to_go = 32
alloc_to_go = 31
alloc_to_go = 30
alloc_to_go = 29
alloc_to_go = 28
alloc_to_go = 27
alloc_to_go = 26
alloc_to_go = 25
alloc_to_go = 24
alloc_to_go = 23
alloc_to_go = 22
alloc_to_go = 21
alloc_to_go = 20
alloc_to_go = 19
alloc_to_go = 18
alloc_to_go = 17
alloc_to_go = 16
alloc_to_go = 15
alloc_to_go = 14
alloc_to_go = 13
alloc_to_go = 12
alloc_to_go = 11
alloc_to_go = 10
alloc_to_go = 9
alloc_to_go = 8
alloc_to_go = 7
alloc_to_go = 6
alloc_to_go = 5

--- Test: Allocation and Release ---
Allocated 1 frame at 513
Released frame 513
Allocated 5 frames starting at 513
Released 5 frames starting at 513
Allocation and release test DONE.

--- Test: Mark Inaccessible Frames ---
Successfully avoided inaccessible frames.
Mark inaccessible test DONE.

--- Test: Zero Frame Allocation ---
Zero frame allocation correctly returned 0.
Zero allocation test DONE.

--- Test: Max Allocation ---
Failed to allocate maximum frames (expected if not enough memory).
Max allocation test DONE.

--- Test: Needed Info Frames ---
Needed info frames for kernel pool: 1
Needed info frames test DONE.
Testing is DONE. We will do nothing forever
Feel free to turn off the machine now.
```

```
phaniyoth11@PhaniGowthar: ~/mp2/MP2_Sources/MP2_Sources$ make run
alloc_to_go = 7
alloc_to_go = 6
alloc_to_go = 5
alloc_to_go = 4
alloc_to_go = 3
alloc_to_go = 2
alloc_to_go = 1
alloc_to_go = 0

--- Test: Allocation and Release ---
Allocated 1 frame at 513
Released frame 513
Allocated 5 frames starting at 513
Released 5 frames starting at 513
Allocation and release test DONE.

--- Test: Mark Inaccessible Frames ---
Successfully avoided inaccessible frames.
Mark inaccessible test DONE.

--- Test: Zero Frame Allocation ---
Zero frame allocation correctly returned 0.
Zero allocation test DONE.

--- Test: Max Allocation ---
Failed to allocate maximum frames (expected if not enough memory).
Max allocation test DONE.

--- Test: Needed Info Frames ---
Needed info frames for kernel pool: 1
Needed info frames test DONE.
Testing is DONE. We will do nothing forever
Feel free to turn off the machine now.
```

Figure 9: Test case outputs showing allocation and edge case handling.